16. Write a program in c to find spanning tree of given graph by using Prims algorithm.

```
#include <stdio.h>
#define INF 1000000
int minKey(int key[], int mstSet[], int n) {
   int min = INF, min_index; for
   (int v = 0; v < n; v++)
      if (mstSet[v] == 0 && key[v] < min) {
         min = key[v]; min_index
         = v;
      } return
   min_index;
} int main()
{
   int   n;   printf(“Abinash   Regmi\n”);
   printf("Enter number of vertices: ");
   scanf("%d", &n); int graph[n][n];
   printf("Enter adjacency matrix (0 for no edge):\n");
   for (int i = 0; i < n; i++)
      for (int j = 0; j < n; j++)
         scanf("%d", &graph[i][j]);
   int parent[n], key[n], mstSet[n];
   for (int i = 0; i < n; i++) {
      key[i] = INF; mstSet[i]
      = 0;
   } key[0] = 0;
   parent[0] = -1;
   for (int count = 0; count < n - 1; count++) {
      int u = minKey(key, mstSet, n);
      mstSet[u] = 1; for (int v =
      0; v < n; v++) {
```

```
            if (graph[u][v] && mstSet[v] == 0 && graph[u][v] < key[v]) {
            parent[v] = u; key[v]
            = graph[u][v];
        }
    } } int total = 0;
  printf("Edges in MST:\n");
  for (int i = 1; i < n; i++) {
      printf("%d - %d : %d\n", parent[i], i, graph[i][parent[i]]); total
      += graph[i][parent[i]];
  }
  printf("Total weight of MST: %d\n", total); return
  0;
}
```

Output 1 :

```
Abinash Regmi:
Enter number of vertices: 4
Enter adjacency matrix (0 for no edge):
0 2 0 6
2 0 3 8
0 3 0 0
6 8 0 0
Edges in MST:
0 - 1 : 2
1 - 2 : 3
0 - 3 : 6
Total weight of MST: 11
```

Output 2:

```
Abinash Regmi:
Enter number of vertices: 5
Enter adjacency matrix (0 for no edge):
0 2 0 6 0
2 0 3 8 5
0 3 0 0 7
6 7 0 0 9
0 5 7 9 0
Edges in MST:
0 - 1 : 2
1 - 2 : 3
0 - 3 : 6
1 - 4 : 5
Total weight of MST: 16
```

Output 3:

```
Abinash Regmi:
Enter number of vertices: 3
Enter adjacency matrix (0 for no edge):
0 1 2
1 0 4
2 4 0
Edges in MST:
0 - 1 : 1
0 - 2 : 2
Total weight of MST: 3
```

17. Write a program in c to find max and min value from given data set by using min-max algorithm.

```
#include <stdio.h>
struct Pair {
   int min, max;
};
struct Pair minmax(int a[], int low, int high) { struct
    Pair p, left, right;
      if (low == high) {
        p.min = p.max = a[low];
        return p;
   } if (high == low + 1) { if
     (a[low] < a[high]) {
           p.min = a[low];
           p.max = a[high];
        } else {
           p.min = a[high];
           p.max = a[low];
        } return p;
   } int mid = (low + high) / 2;
   left = minmax(a, low, mid); right
   = minmax(a, mid + 1, high);
   p.min = (left.min < right.min) ? left.min : right.min;
   p.max = (left.max > right.max) ? left.max : right.max;
return p; } int main() { int n;
   printf("Abinash Regmi:\n");
   printf("Enter number of elements:
   "); scanf("%d", &n); int a[n];
   printf("Enter elements:\n"); for (int i
   = 0; i < n; i++)
       scanf("%d", &a[i]);
```

```
    struct Pair result = minmax(a, 0, n - 1);
    printf("Minimum: %d\n", result.min);
    printf("Maximum: %d\n", result.max);
    return 0;
}
```

Output 1:

```
Enter number of elements: 5
Enter elements:
3 1 9 7 2
Minimum: 1
Maximum: 9
```

Output 2:

```
Abinash Regmi:
Enter number of elements: 4
Enter elements:
10 20 5 8
Minimum: 5
Maximum: 20
```

Output 3:

```
Abinash Regmi:
Enter number of elements: 3
Enter elements:
-2 -5 -1
Minimum: -5
Maximum: -1
```

18. Write a program in c to select i^{th} smallest element by using selection in expected linear time algorithm.

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h> void
swap(int *a, int *b) {
   int t = *a; *a = *b; *b = t;
}
int partition(int a[], int l, int h) {
   int p = a[h], i = l; for (int j
   = l; j < h; j++) { if (a[j] <=
   p) {
         swap(&a[i], &a[j]); i++;
      } }
   swap(&a[i], &a[h]);
return i; }
int randPartition(int a[], int l, int h) {
   int r = l + rand() % (h - l + 1);
   swap(&a[r], &a[h]); return
   partition(a, l, h);
}
int quickSelect(int a[], int l, int h, int k) {
   if (l <= h) {
      int pi = randPartition(a, l, h);
      if (pi == k)
         return a[pi];
      else if (pi > k)
         return quickSelect(a, l, pi - 1, k);
      else
         return quickSelect(a, pi + 1, h, k);
```

```
    } return -
1; } int
main() {
    int n, i, k; printf("Abinash Regmi
    :\n"); printf("Enter number of
    elements: "); scanf("%d", &n); int
    a[n]; printf("Enter elements:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    printf("Enter i (1 to %d): ", n); scanf("%d", &k);
    srand(time(0)); int result = quickSelect(a, 0, n -
    1, k - 1); printf("%dth smallest element: %d\n",
    k, result); return 0;
}
```

Output 1:

```
Abinash Regmi:
Enter number of elements: 5
Enter elements:
7 2 8 4 1
Enter i (1 to 5): 3
3th smallest element: 4
```

Output 2:

```
Abinash Regmi:
Enter number of elements: 6
Enter elements:
10 3 5 8 2 6
Enter i (1 to 6): 2
2th smallest element: 3
```

Output 3:

```
Abinash Regmi:
Enter number of elements: 4
Enter elements:
12 7 9 3
Enter i (1 to 4): 1
1th smallest element: 3
```

19. Write a program in c to select subset sum by using backtracking.

```
#include <stdio.h> int
x[20]; // solution vector
void subset(int w[], int n, int sum, int k, int curr) {
   x[k] = 1; if (curr + w[k] ==
   sum) {
      printf("Subset: ");
      for (int i = 0; i <= k; i++) if
         (x[i] == 1)
            printf("%d ", w[i]);
      printf("\n");
   }
   else if (curr + w[k] < sum) {
      subset(w, n, sum, k + 1, curr + w[k]);
   } if (curr < sum)
   {
      x[k] = 0;
      subset(w, n, sum, k + 1, curr);
   } }
int main() {
   int n, sum , w[n]; printf("Abinash
   Regmi :\n"); printf("Enter number of
   elements: "); scanf("%d", &n);
   printf("Enter elements:\n"); for (int i
   = 0; i < n; i++)
      scanf("%d", &w[i]);
   printf("Enter required sum: ");
   scanf("%d", &sum); subset(w,
   n, sum, 0, 0); return 0;
}
```

Output 1:

```
Abinash Regmi:
Enter number of elements: 3
Enter elements:
1 2 3
Enter required sum: 3
Subset: 1 2
Subset: 3
```

Output 2:

```
Abinash Regmi:
Enter number of elements: 5
Enter elements:
3 5 6 7 8
Enter required sum: 11
Subset: 3 8
Subset: 5 6
```

Output 3:

```
Abinash Regmi:
Enter number of elements: 4
Enter elements:
2 4 6 8
Enter required sum: 10
Subset: 2 8
Subset: 4 6
```

20. Write a program in c to select simulate N-queen problem by using backtracking.

```
#include <stdio.h> #include
<stdlib.h> int x[20]; // stores
column positions int place(int k, int
i) {
   for (int j = 1; j < k; j++) {
      if (x[j] == i || abs(x[j] - i) == abs(j - k))
         return
0; } return 1; }
void nqueen(int k, int n) {
   for (int i = 1; i <= n; i++) {
      if (place(k, i)) {
         x[k] = i; if (k
         == n) {
            printf("Solution: ");
            for (int j = 1; j <= n; j++)
               printf("%d ", x[j]);
            printf("\n");
         } else { nqueen(k +
            1, n);
         } } } }
int main() {
   int n;
   printf("Abinash Regmi:\n");
   printf("Enter number of queens:
   "); scanf("%d", &n); nqueen(1, n);
   return 0;
}
```

Output 1:

```
Abinash Regmi:
Enter number of queens: 4
Solution: 2 4 1 3
Solution: 3 1 4 2
```

Output 2:

```
Abinash Regmi:
Enter number of queens: 5
Solution: 1 3 5 2 4
Solution: 1 4 2 5 3
Solution: 2 4 1 3 5
Solution: 2 5 3 1 4
Solution: 3 1 4 2 5
Solution: 3 5 2 4 1
Solution: 4 1 3 5 2
Solution: 4 2 5 3 1
Solution: 5 2 4 1 3
Solution: 5 3 1 4 2
```

Output 3:

```
Abinash Regmi:
Enter number of queens: 4
Solution: 2 4 1 3
Solution: 3 1 4 2
```